

Sesión 3

gRPC para microservicios de ML



Comunicación interna de baja latencia y contratos estrictos entre servicios.

Agenda de hoy

0:00		Recap y encuadre Dónde estamos en la plataforma
0:10		El problema de comunicar servicios Sockets, capas y por qué duele
0:35		RPC: la idea Llamar a un procedimiento remoto como si fuera local
1:00		Serialización y Protocol Buffers IDL, tipado y por qué es tan compacto
1:30		¿Qué es gRPC? HTTP/2, los 4 tipos de RPC y arquitectura
1:55		gRPC vs REST vs GraphQL Cuándo conviene cada uno · gRPC en MLOps
2:15		Práctica proto → stubs → servidor → cliente → streaming
2:50		Mini-TP 3 y cierre Tu modelo servido por gRPC

La plataforma ya expone REST y GraphQL; hoy hablamos entre servicios

REST

GraphQL

gRPC

Streaming

Cloud/Lake

Federado

Seguridad

Integra

En las Sesiones 1 y 2

- REST (FastAPI): el borde externo, universal
- GraphQL: consultas flexibles de metadatos y linaje
- Ambos sobre HTTP/1.1 + JSON (texto)

Hoy resolvemos

- Dos servicios internos necesitan hablar con latencia mínima
- Con un contrato estricto y tipado, y streaming
- JSON sobre HTTP/1.1 se queda corto → gRPC

El problema de fondo

Para coordinarse, los servicios deben comunicarse. Si cada aplicación reimplementa la comunicación para cada medio de transporte, no escala ni se mantiene.



Reimplementar todo

¿Reescribir cada app para cada nuevo medio de transmisión?



Heterogeneidad

Distinto hardware, sistema operativo, lenguaje, orden de bytes.



Fallos y cambios

La red falla; los cambios de un lado no deben romper al otro.



Necesitamos abstraer

Una capa que oculte los detalles y ofrezca una interfaz simple.

De las capas a los sockets

La solución: capas

Cada capa ofrece abstracciones a la de arriba. Una nueva aplicación solo se implementa contra la interfaz, no contra el medio.

- Aplicación
- Transporte (TCP/UDP) — proceso a proceso
- Red (IP) — entrega entre redes
- Enlace — host a host

TCP/IP: entrega de paquetes entre hosts distantes.

Sockets: la interfaz de red

El socket es la puerta que el sistema operativo da a la red: intercambio explícito de mensajes entre procesos.

Pero son de bajo nivel:

- Fuerzan un modelo de leer/escribir bytes
- Programar es más natural con una interfaz de funciones
- Cada app repite serialización, direcciones y fallos

La idea: llamar a lo remoto como si fuera local

RPC (Remote Procedure Call) oculta los detalles de la comunicación detrás de una llamada a función. El cliente invoca un procedimiento; parece local, pero se ejecuta en otro proceso o máquina.



El stub (proxy) tiene la MISMA interfaz que la función remota: ordena parámetros, los envía y devuelve el resultado. La respuesta vuelve por el mismo camino.

Retos que RPC resuelve por ti: binding (localizar el servicio), interoperabilidad (orden de bytes, tipos) y fallos de red.

Stubs, dispatcher e IDL

Los dos stubs

Stub de cliente (proxy): tiene la interfaz de la función; ordena (marshal) los parámetros y llama al servidor.

Stub de servidor:

- Despachador: recibe la solicitud e identifica el método.
- Esqueleto: desempaqueta (unmarshal) y llama a la función real.

IDL: describe una vez

El Interface Definition Language especifica los procedimientos remotos (nombres, parámetros, retornos). Un compilador genera los stubs de cliente y servidor.

```
// IDL (idea)
service Scoring {
  rpc Predict(Features)
    returns (Prediction);
}
```

Representar datos entre máquinas heterogéneas

En remoto no hay memoria compartida: hay que convertir las estructuras a una secuencia de bytes (marshalling) y reconstruirlas del otro lado. Sin punteros, y con un formato acordado.



Orden de bytes

Big-endian vs little-endian; enteros y coma flotante de distinto tamaño.



Sin punteros

Una dirección de memoria no significa nada en la otra máquina.



Formato acordado

Emisor y receptor deben interpretar los bytes igual (un esquema).

Ejemplos históricos: XDR, XML, JSON... y Protocol Buffers, el que usa gRPC.

Formatos: XML vs JSON vs Protocol Buffers

Formato	Cómo es	Costo
XML	Texto, muy verboso, con esquema	Grande y lento de parsear
JSON	Texto ligero, legible, sin tipos	Compacto vs XML, pero sigue siendo texto
Protocol Buffers	Binario, tipado, con esquema (.proto)	Muy pequeño y muy rápido

Protobuf, en números

3–10×

más pequeño que XML/JSON

20–100×

más rápido de (de)serializar

~28 B

un mensaje que en XML son ≥ 69 B

El contrato: mensajes y servicios tipados

```
syntax = "proto3";

message Features {
  repeated double values = 1;
  string model = 2;
}
message Prediction {
  double score = 1;
  string model_version = 2;
}
service Scoring {
  rpc Predict(Features) returns (Prediction);
}
```

Qué mirar

- Cada campo tiene un número (su etiqueta en el binario)
- Tipado fuerte: el esquema valida entrada y salida
- Extensible: agregar campos mantiene compatibilidad
- Independiente del lenguaje: genera código para 10+

¿Qué es gRPC?

Un framework RPC moderno y de código abierto (nacido en Google, sucesor de su sistema interno Stubby, que mueve $\sim 10^{10}$ RPC por segundo). Facilita construir sistemas distribuidos heterogéneos.



Contrato en .proto

IDL con Protocol Buffers: describe el servicio una vez.



Genera stubs

protoc crea cliente y servidor en 10+ lenguajes.



Sobre HTTP/2

Multiplexado, binario, con streaming en ambas direcciones.



Seguro por diseño

Integración con SSL/TLS para autenticar y cifrar.

HTTP/2: por qué es el transporte

HTTP/1.1

- Solicitud–respuesta, en orden (sin paralelismo real)
- Bloqueos 'head-of-line' entre solicitudes
- Se necesitan muchas conexiones para no trabarse
- Texto, cabeceras repetidas y pesadas

HTTP/2 (gRPC)

- Una conexión TCP con múltiples streams multiplexados
- Framing binario compacto (no texto)
- Compresión de cabeceras y control de flujo
- Server push y streaming bidireccional nativo

Los cuatro tipos de llamada



Unary

1 request → 1 response. La llamada clásica: predecir un caso.



Server streaming

1 request → N responses. El servidor va emitiendo resultados.



Client streaming

N requests → 1 response. El cliente envía un lote y recibe un resumen.

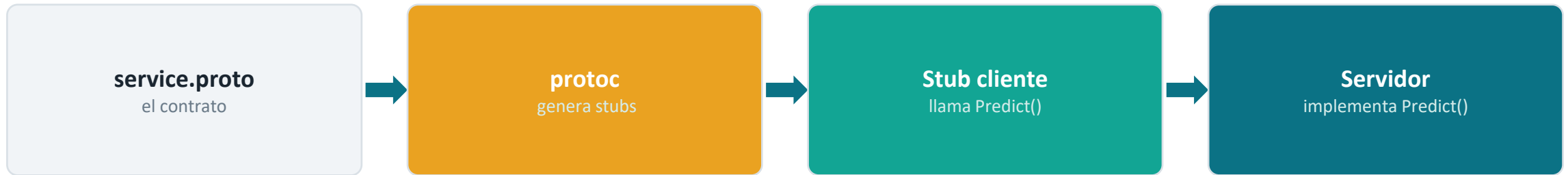


Bidirectional

N ↔ N. Ambos envían streams a la vez (scoring en vivo).

Arquitectura: del .proto al servicio

El .proto es la única fuente de verdad. protoc genera el stub del cliente y la clase base del servidor; ambos hablan por HTTP/2.



Cliente y servidor se comunican por un canal HTTP/2. El stub hace que la llamada remota se vea como una función local y tipada.

gRPC vs REST vs GraphQL, sin exageraciones

Aspecto	REST	GraphQL	gRPC
Transporte	HTTP/1.1	HTTP/1.1	HTTP/2
Formato	JSON (texto)	JSON (texto)	Protobuf (binario)
Contrato	OpenAPI (opcional)	Esquema/SDL	*.proto (obligatorio)
Streaming	Limitado	Subscriptions	Nativo, bidireccional
Latencia	Media	Media	Muy baja
Mejor para	Borde externo	Consultas flexibles	Servicio a servicio interno

Regla práctica: REST/GraphQL de cara al mundo; gRPC entre tus microservicios, donde mandan latencia y contrato.

Para qué lo usamos



Inferencia interna

Un servicio de features llama al de scoring con latencia mínima.



Streaming de predicciones

Enviar un flujo de eventos y recibir scores a medida que llegan.



Microservicios de ML

Preproceso, modelo y postproceso como servicios que se llaman entre sí.



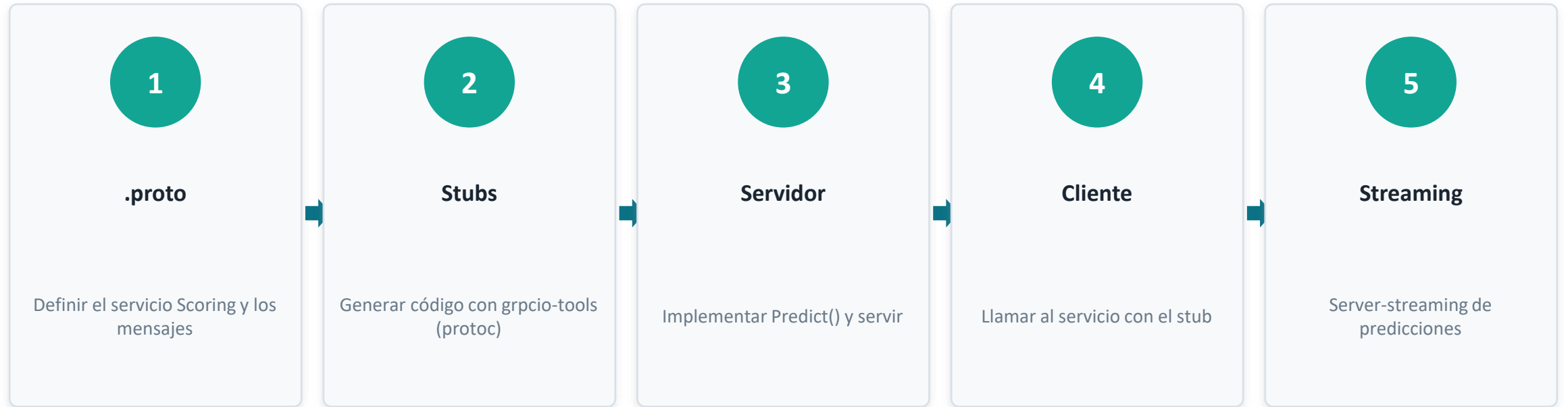
Contrato estricto

El .proto evita 'garbage in': tipos y forma fijados de antemano.

Nota: el navegador no habla gRPC directo; para el borde externo se usa gRPC-Web o una pasarela. Internamente, gRPC puro.

Qué construimos

Un servicio de scoring por gRPC de punta a punta: del contrato .proto al cliente, con una llamada unary y una de streaming. Todo con uv; el código está en clase3/Practica.



NOTEBOOK `grpc_tutorial.ipynb`

El .proto y la generación de stubs

```
// scoring.proto
syntax = "proto3";
service Scoring {
  rpc Predict(Features) returns (Prediction);
  rpc PredictStream(Features)
    returns (stream Prediction);
}
```

```
# generar stubs (crea *_pb2.py y *_pb2_grpc.py)
uv add grpcio grpcio-tools
uv run python -m grpc_tools.protoc -I. \
  --python_out=. --grpc_python_out=. scoring.proto
```

NOTEBOOK `grpc_tutorial.ipynb`

Qué mirar

- Un método normal y uno con 'stream'
- protoc genera mensajes y stubs
- *_pb2.py: los mensajes
- *_pb2_grpc.py: cliente y servidor
- No se edita el código generado

El servidor

```
import grpc, joblib
from concurrent import futures
import scoring_pb2, scoring_pb2_grpc

model = joblib.load('model.pkl')
class Scoring(scoring_pb2_grpc.ScoringServicer):
    def Predict(self, request, context):
        y = model.predict([request.values])[0]
        return scoring_pb2.Prediction(
            score=float(y), model_version='v1')

srv = grpc.server(futures.ThreadPoolExecutor())
scoring_pb2_grpc.add_ScoringServicer_to_server(...)
srv.add_insecure_port('[::]:50051'); srv.start()
```

Qué mirar

- El modelo se carga UNA vez al iniciar
- Heredas de la clase base generada
- Puerto típico de gRPC: 50051
- ThreadPool para atender en paralelo

NOTEBOOK `grpc_tutorial.ipynb`

Cliente y streaming

```
import grpc, scoring_pb2, scoring_pb2_grpc

ch = grpc.insecure_channel('localhost:50051')
stub = scoring_pb2_grpc.ScoringStub(ch)

# unary: 1 -> 1
r = stub.Predict(scoring_pb2.Features(
    values=[0.2,1.3,0.7], model='churn'))
print(r.score, r.model_version)

# server streaming: 1 -> N
for r in stub.PredictStream(features):
    print(r.score)
```

NOTEBOOK `grpc_tutorial.ipynb`

Qué mirar

- El stub hace la llamada como si fuera local
- `insecure_channel` para local; TLS en producción
- El streaming se recorre como un iterador
- Mismo contrato en cliente y servidor

Errores comunes (y cómo evitarlos)



Editar el código generado

_pb2.py se regeneran: no los toques, regénalos.



Reimportar sin regenerar

Si cambias el .proto, vuelve a correr protoc.



Cargar el modelo por request

Cárgalo una vez al iniciar el servidor, no en Predict().



Sin manejo de errores

Usa context.set_code/set_details para errores claros.



Canal por llamada

Reutiliza el channel; crearlo cada vez es caro.



Sin TLS en producción

insecure_channel solo para local; usa SSL/TLS al desplegar.



MINI-TP 3 · INDIVIDUAL · ENTREGA ESTA SEMANA

Sirve tu modelo por gRPC

Consigna

- Escribe un `scoring.proto` con un servicio para tu modelo (mensajes de entrada y salida tipados).
- Genera los stubs y implementa el servidor cargando tu modelo una sola vez.
- Llama al servicio desde un cliente Python (unary).
- Agrega un método de server-streaming que puntúe un lote.
- Compara la latencia contra tu endpoint REST de la Sesión 1.

Se evalúa

- Corre de punta a punta
- El `.proto` tipa entrada y salida
- Unary + streaming funcionando
- Reflexión gRPC vs REST (latencia)

Actividad: `mini_tp3_actividad.ipynb` (starter con celdas TODO).

Dónde quedó la plataforma

REST

GraphQL

gRPC

Streaming

Cloud/Lake

Federado

Seguridad

Integra

Hoy logramos

- Servimos el modelo por gRPC (unary + streaming)
- Contrato .proto tipado y binario sobre HTTP/2
- Sabemos cuándo gRPC gana a REST/GraphQL



Próxima · Sesión 4

Streaming e inferencia en tiempo real: cuando los datos no vienen de a uno sino como un flujo continuo. Y arranca el TP integrador (Hito #1: equipos + arquitectura).

Para la próxima: ten tu Mini-TP 3 corriendo y piensa el equipo del TP integrador.

Referencias y recursos

Herramientas y código

- gRPC — grpc.io/docs · Protocol Buffers — protobuf.dev · paquetes: `uv add grpcio grpcio-tools`
- Python gRPC — grpc.io/docs/languages/python (Quick start, Basics tutorial).
- Repositorio del curso: `clase3/Practica` (`grpc_tutorial.ipynb`, `mini_tp3_actividad.ipynb`).

Lectura

- Documentación de gRPC (motivación, arquitectura, HTTP/2, autenticación).
- Designing Machine Learning Systems — Chip Huyen (O'Reilly): model serving y microservicios.